Computational Statistics - Coursework

MSc in Statistics 2025/26, Imperial College London

06046958

I, 06046958, certify that this assessed coursework is my own work, unless otherwise acknowledged, and includes no plagiarism. I have not discussed my coursework with anyone else except when seeking clarification with the module lecturer via email or on MS Teams. I have not shared any code underlying my coursework with anyone else prior to submission.

Question 1

(a)

We discuss three cases for $\sigma = (0.1, 1, 10)^T$ and choose the start points at $x_0 = (2, 20, 30)^T$, and display the results respectively:

```
Chain 1 : Sigma is 1 , ESS is: 85246.48
Chain 2 : Sigma is 1 , ESS is: 59177.38
Chain 3 : Sigma is 1 , ESS is: 174623.1
Chain 4 : Sigma is 5 , ESS is: 388.6708
Chain 5 : Sigma is 5 , ESS is: 211.1964
Chain 6 : Sigma is 5 , ESS is: 363.447
Chain 7 : Sigma is 8 , ESS is: 968.6788
Chain 8 : Sigma is 8 , ESS is: 1147.484
Chain 9 : Sigma is 8 , ESS is: 1133.775
```

And the traceplots:

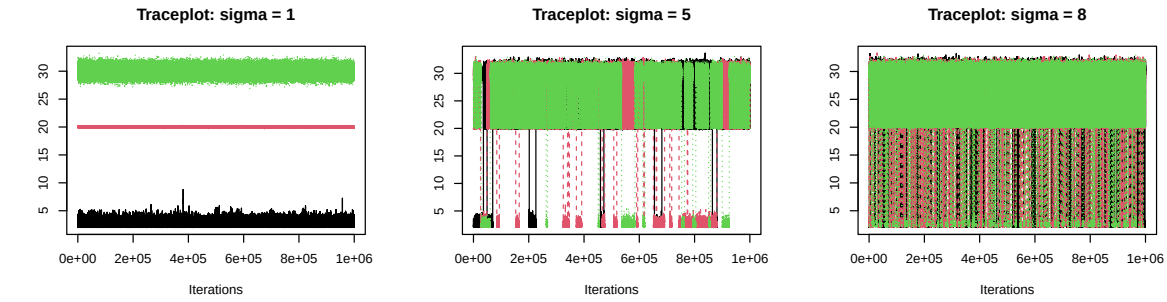


Figure 1: Traceplot for three different variance

```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]         46        157

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       1.13       1.38

Potential scale reduction factors:

     Point est. Upper C.I.
[1,]          1          1
```

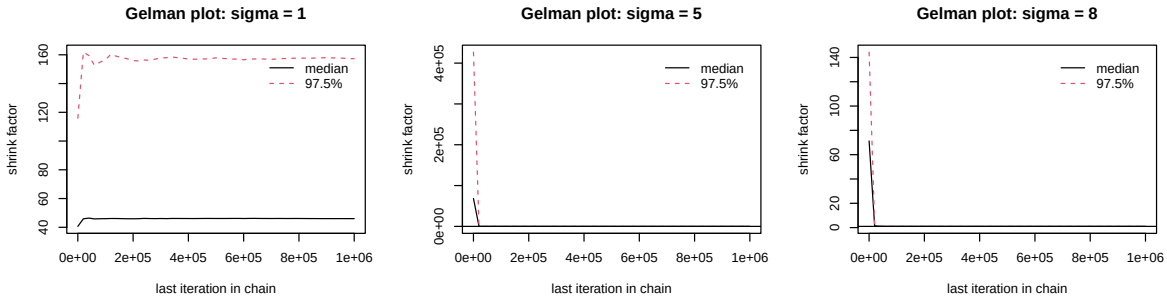


Figure 2: Gelman plots for three different variance

Some comments:

- Mixing quality: Visually, when $\sigma = 1$, the Markov Chains (MCs) trapped in the neighborhood of their start point, but they other two cases could jump out of other modes. However, the cases for high σ will freeze at their stages for a long time. On the other hand, when looking at the metric **ESS**, the cases for $\sigma = 5, 8$ show extremely low statistics, implying low autocorvariance and efficiency, but the case of $\sigma = 2$ has good performance in different starts points. In conclusion, the mixing quality is less satisfactory in both 3 cases.
- Convergence: For $\sigma = 1$, the MCs are trapped in their modes visually, and the "R-hat" is extremely high, implying no convergence. As for the other two cases, the "R-hat" for $\sigma = 5$ is closed but still larger than 1.1, and that for $\sigma = 8$ is less than 1.1. And traceplot provide some good evidence that they are convergent, but we need to plot more samples to see whether their histograms coincide with the plot of the density.

(b)

I will use truncated normal distribution as proposal. Consider

$$1 - \Phi(\frac{2-\mu}{\sigma}) = \int_2^\infty \mathcal{N}(x; \mu, \sigma^2)dx,$$

it possible for us to define the truncated density:

$$q(x|y) = \frac{\mathcal{N}(x; y, \sigma^2)}{1 - \Phi(\frac{2-y}{\sigma})}\mathbb{1}_{\{x\geq 2\}},$$

which guranteens the proposal must be larger than 2 almost surely, so that it must be in the support of f. Consider the acceptance rate, we have:

$$\alpha(x, y) = \frac{f(y)q(x|y)}{f(x)q(y|x)} = \frac{f(y)(1 - \Phi(\frac{2-x}{\sigma}))}{f(x)(1 - \Phi(\frac{2-y}{\sigma}))}\mathbb{1}_{\{x\geq 2, y\geq 2\}} \Rightarrow \alpha(x, y) = \min\left(1, \frac{f(y)(1 - \Phi(\frac{2-x}{\sigma}))}{f(x)(1 - \Phi(\frac{2-y}{\sigma}))}\mathbb{1}_{\{x\geq 2, y\geq 2\}}\right).$$

Due to the symmetry, we could verify that based on this proposal distribution, the MC satisfies the detailed balance condition, so that the MC will converge to the invariant distribution.

```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       39.5       86.5
```

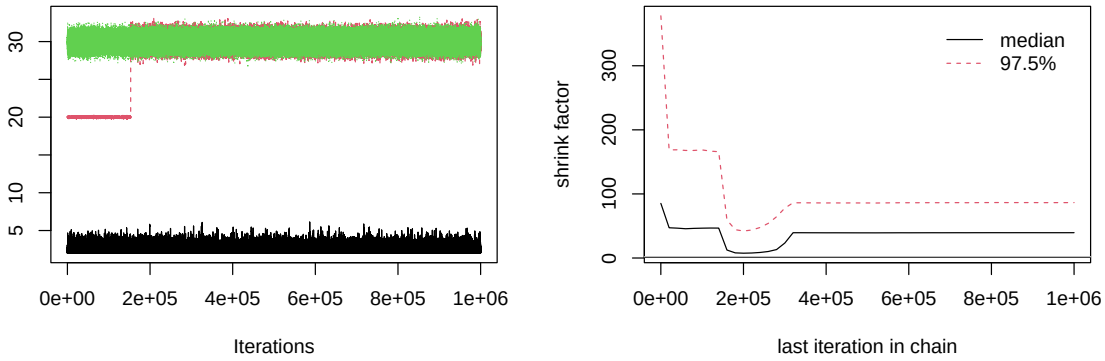


Figure 3: Improved MH-algotirhm by restricting the proposal in the support

Conclusion: Even though we restrict the proposal y in the support, the MCs still cannot converge due to the multimodality. Visually, any three chains cannot jump out the neighborhood of their start points (modes), at least in 1×10^5 steps. Also, the "R-hat" metric is 46.1, far larger than the baseline 1.1, providing stronger evidence to reject the chains are convergent.

(c)

Pseudo Codes

- Settings: α_0, σ, T_s.
- Draw u from unif [0,1].
- If $u \leq \alpha_0$, then we do MCMC in the chain and update X_{i+1} from X_i. I choose the random walk ($\mathcal{N}(y; x, \sigma_i^2)$) as the proposal distribution, so the acceptance rate is

$$\alpha_1(x, y) = \min\{1, \frac{f(y)}{f(x)}\}.$$

- Else, we swap the chain: by randomly choosing one neighbor chain (e.g. $(X_i^n, X_{i+1}^n) \to (X_{i+1}^{n+1}, X_i^n)$), this proposal is symmetric, so we accept this proposal with probability

$$\alpha_2 = \min\{1, \frac{f^{1/T_{i+1}}(X_i^n) f^{1/T_i}(X_{i+1}^n)}{f^{1/T_{i+1}}(X_{i+1}^n) f^{1/T_i}(X_i^n)}\}.$$

In terms of the transition kernel, for $u \leq \alpha_0$, we have

$$P_{x,y} = q(y|x)\alpha_1(x,y) = \mathcal{N}(y; x, \sigma_i^2) \cdot \alpha_1(x,y).$$

On the other hand, as we consider the transition between the joint distribution, suppose we have m chains, the transition kernel is

$$P = \Pi(\textit{proposed state})q(\textit{proposed state}|\textit{current state}) = \frac{1}{m-1}\alpha_2.$$

In my experiment, I used $\alpha_0 = 0.5$, $T_s = (1, 20, 100, 200, 500)^T$, $\sigma = (0.1, 0.5, 1, 2, 5)$, and ran 5 chains.

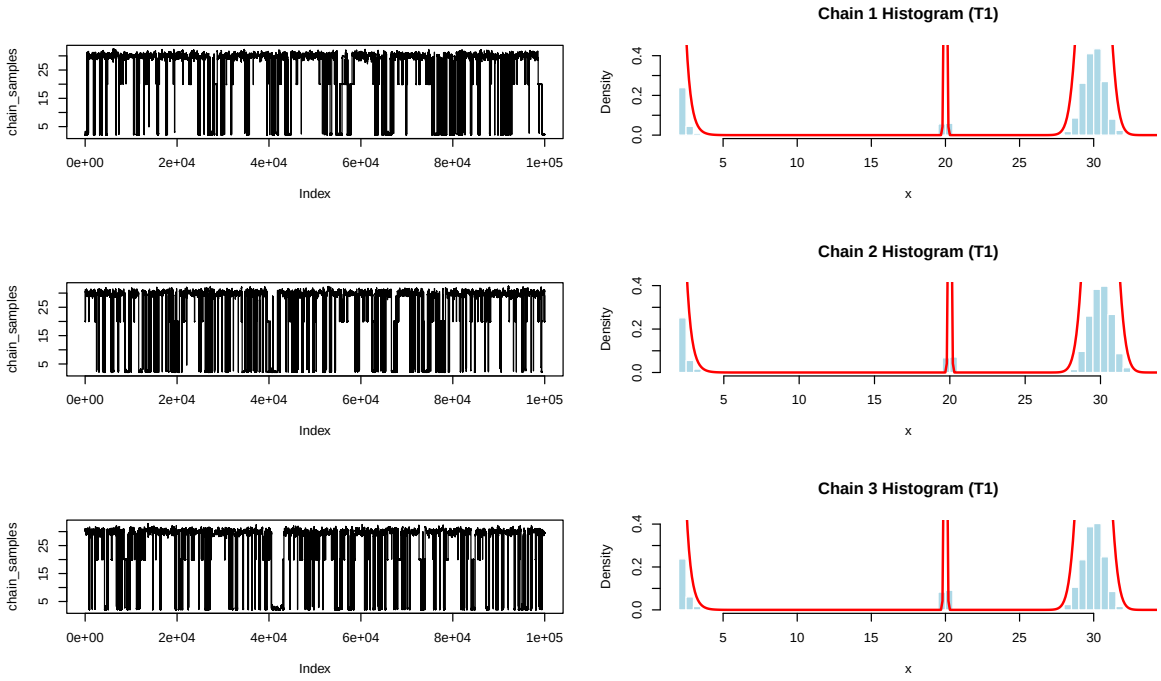
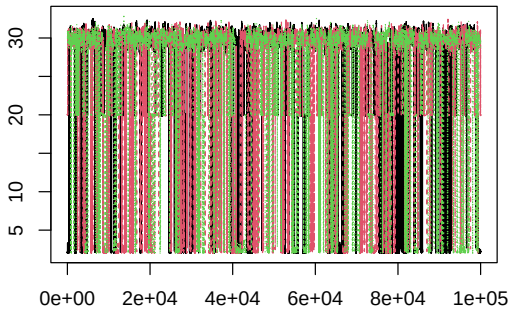
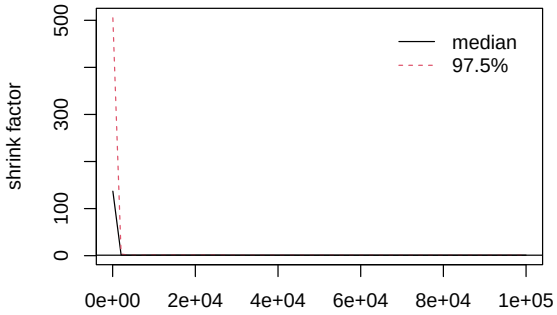


Figure 4: Compare with the true density

We then do the convergence diagnostic for this chain,






```
Potential scale reduction factors:

     Point est. Upper C.I.
[1,]       1.01       1.02
```

From the diagonize of the cold chain, namely, $f(x)^{1/1} = f(x)$, the mixing quality is really well and metric "R-hat" is smaller than 1.1, there is a strong evidence the chain is convergent. Look back to the histogram against density plot, the frequency plot fits well to the density, which also provides strong evidence that the chain is convergent and is not sensitive to the start point.

(d)

Convergence

In 1(a), I used random walk to propose y, in 1(b) I restricted propose y in the support of f. Although the choice of large σ may result in low "R-hat", there still needs much more steps to see. In the typical cases, the MCs cannot get rid of the neighborhood of modes for a long time, and the "R-hat" is extremely high, indicating no convergence in both 1(a) and 1(b). However, by implementing parallel tempering in 1(c), I choose start points on purpose at the modes of f, the MCs can search all the support of f. The frequency plot also well fits to the density of f, as well as the "R-hat" is less than 1, suggesting very good convergence. Hence, parallel tempering is a good method to tackle the multimodality problem.

Mixing quality

The mixture quality of 1(a) and 1(b) is bad. From the traceplot, when σ is small, the MC will only move in the neighborhood of its start point and it is apparent to see different chains

cannot mix together, although ESS is high when σ is small, suggesting the potential to mix in long run. When σ is large, the mixing quality is not satisfactory either. The quality of 1(c) is really good visually from the traceplot.

Computational cost

Compare to random walk proposal in 1(a), the truncated proposal I used in 1(b) is much more efficient as it is possible for random walk to generate proposals that is out of the support of the f and these samples cannot be accepted, but the truncated proposal is not. However, these two proposals cannot produce convergent sample MC in $N = 1000000$ steps, which is extremely inefficient compared to parallel tempering in 1(c). Due to the multimodaility, the MC will propose an extreme value (e.g. from 20 to 2), whose probability is small, to jump. In addition to this, the ESS is low in all cases in 1(a) and 1(b), suggesting very small effective samples. The method in 1(c) is robust in the start point and can achieve convergence with the same baseline with in 1(a) and 1(b). And from the Gelman plot for "R-hat", the metric converges very quickly in less than 1000 samples, suggesting strong efficiency.

(e)

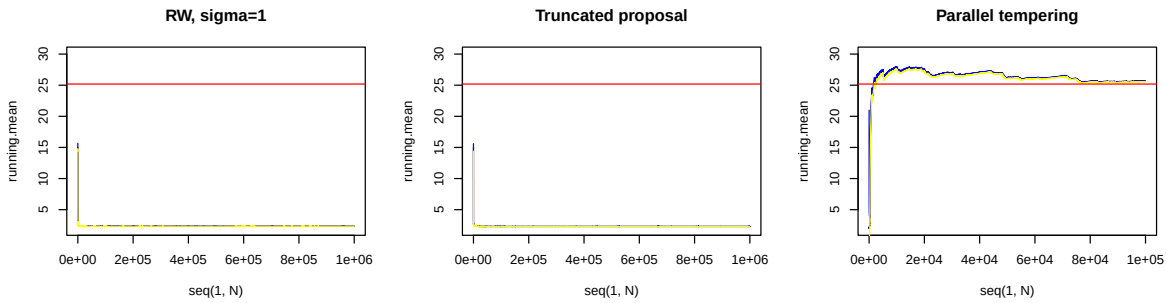
We can easily figure out $\mathbb{E}(X)$ via R, which is 25.18417.

We directly use samples generated by Markov chain to figure out $\mathbb{E}(X)$.

```
The estimator by chain 1 is: 2.335298 and ci is: [ 1.671585 , 2.99901 ]

The estimator by chain 2 is: 2.302984 and ci is: [ 1.694212 , 2.911756 ]

The estimator by chain 3 is: 25.63622 and ci is: [ 6.649492 , 44.62296 ]
```



Conclusion

We spot that the convergent to $\mathbb{E}(X)$ is positive correlated to the convergent to the MCs. As indicated in previous argument, the MC in 1(a) and 1(b) cannot converge to the whole

distribution, so that they will cause severe errors between the true value. But 1(c) can explore all values with target distribution, hence provides good approximation to $\mathbb{E}(X)$.

Question 2

(a)

(i)

Let me explain how the optimization is performed. We notice that our loss function is not differentiable at q. So to avoid some incidence, I choose to use "Nelder-Mead" rather than quicker method "BFGS". The trival vector $(0, 0, 0)$ is chosen as my start point. And the optimization converge ultimately.

```
When tau is 0.1 ,the empirical loss is 0.6964838 and the estimator is -
1.229301 -0.7937421 4.543845 convergence: 0
When tau is 0.5 ,the empirical loss is 1.904715 and the estimator is -11.42951 -
0.5723921 6.43675 convergence: 0
When tau is 0.9 ,the empirical loss is 1.133979 and the estimator is -27.63414 -
0.200334 9.303498 convergence: 0
```

(ii)

The output of empirical loss of test data set is stated as follow:

```
When tau is 0.1 ,the empirical loss is: 0.7835174
When tau is 0.5 ,the empirical loss is: 1.977712
When tau is 0.9 ,the empirical loss is: 1.245324
```

(iii)

Let's have a recap, the choice of τ is $(0.1, 0.5, 0.9)$. And the displayed results in my experiments are very close to the initial setting, implying some effectiveness of our method.

```
[1] 0.09210526 0.48026316 0.88815789
```

(b)

(i)

Let me explain how the optimization is performed. We notice that our loss function is not differentiable at q. So to avoid some incidence, I choose to use "Nelder-Mead" rather than quicker method "BFGS". The trival vector $(0, 0, 0, 0, 0, 0)$ is chosen as my start point as we have six parameters. And the optimization doesn't converge ultimately.

```
When tau is 0.1 ,the empirical loss is 0.6583881 and the estimator is -
1.017769 0.3667278 -0.0528613 0.004520817 0.706361 -0.1956024 convergence: 1
When tau is 0.5 ,the empirical loss is 1.880985 and the estimator is -0.2010836 -
0.9668291 -0.1125912 0.01950876 0.7236458 0.03394346 convergence: 1
When tau is 0.9 ,the empirical loss is 0.8436352 and the estimator is -
0.472779 1.760224 0.08120124 0.00781569 0.8769808 -0.421138 convergence: 1
```

(ii)

```
When tau is 0.1 ,the empirical loss is: 1.970295
When tau is 0.5 ,the empirical loss is: 17.65133
When tau is 0.9 ,the empirical loss is: 7.628274
```

```
[1] 0.09210526 0.46052632 0.87500000
```

(c)

Given that $\lambda \in [10^{-4}, 10^{-1}]$, we choose pick 100 samples, where the neighbor distance is 0.001.

The optimization setting is the same as that in 2(b). And we spot that very few training procedures converge when $\tau = 0.1, 0.5$. But it improves when $\tau = 0.9$.

```
About 2  percent of experiments converge in the training
When tau is 0.1 , the best lambda is 0.0111
About 3.8  percent of experiments converge in the training
When tau is 0.5 , the best lambda is 0.0051
About 24.2  percent of experiments converge in the training
When tau is 0.9 , the best lambda is 0.0081
```

```
The empirical loss is 0.6643584 the coefficients are 1.905407 1.779397 -
0.002839653 -0.006681427 0.6250297 -0.3886277 when tau is 0.1 Convergence: 1
The empirical loss is 1.598406 the coefficients are -0.1990285 2.544147 1.054229 0.002767224
0.5768584 when tau is 0.5 Convergence: 1
The empirical loss is 1.046116 the coefficients are 1.401273 3.032821 2.258348 0.006443642 0
0.6807364 when tau is 0.9 Convergence: 1


When tau is 0.1 ,the empirical loss is: 0.7580092
When tau is 0.5 ,the empirical loss is: 1.847137
When tau is 0.9 ,the empirical loss is: 0.9663356


[1] 0.1118421 0.5723684 0.9210526
```

(d)

Coefficient magnitudes:

— Compare Model(b) and Model(c)

Due to the penalty term in Model(c), it means that our model will optimize the loss as well as reduce the magnitude of β.

Test loss:

- Compare Model(a) and Model(b)

As Model(b) introduces more variables than Model(a), it is possible that Model(b) contains too much information. And the testing lossse show that in all value of τ, the testing losses of Model(b) are much higher than those of Model(b), especially then $\tau = 0.5, 0.9$. From the convergence in Model(b), the more parameters make it harder to converge an optimal parameters, trapping in the local optimization and leading to high testing loss.

- Compare Model(b) and Model(c)

Model(b) and Model(c) shares the same number of parameter, but Model(c) introduces ridge regression. In all three cases of τ, the testing losses of Model(c) is much smaller than those of Model(b).

Overfitting:

- Model(a) The training loss and the testing loss is very close in both three cases, suggesting a good fitting.
- Model(b) Although the testing loss is about 2 for $\tau = 0.1$, it is extremely large when $\tau = 0.5, 0.9$, but the training loss is low, suggesting overfitting in this model.

- Model(c) Although the loss is not convergent in training, compared to the Model(b), the difference between the training and testing loss is low, suggesting no overfitting in this model. Also, it is a large improvement compared to the Model(b).

Empirical coverage:

- Model(a)

The result is $(0.09210526, 0.48026316, 0.88815789)$, very closed to our baseline. As we mentioned before, this model fits well.

- Model(b)

The result is $(0.09210526, 0.46052632, 0.87500000)$, although the first and the third item are very closed to the baseline, the middle item, with the largest testing loss, is far away from our baseline.

- Model(c)

The result is $(0.1118421, 0.5723684, 0.9210526)$, except the middle part, of the estimated quantiles are close to our baseline. The reason perhaps is the non-convergence of the training part.

(e)

(i)

Not necessarily, it is not guaranteed. Firstly, in the optimization procedure, the models in (b) and (c) don't converge to optimal solutions, so that it may cause large bias in estimating the true parameter and lead to the crossing of other quantile models. Secondly, there is no restriction of forcing one model to be larger than the other, as we didn't embed any another properties to this family of models. Optimize independently may be an another cause.

(ii)

Rather than optimize independently, we attempt to optimize this family of models together. We also need to add some penalty if our models against the quantile rule, so we may do similar what we do in ridge regression model.

$$R_{\lambda}^{Ridge}(\beta) = \frac{1}{n_{train}} \sum_{\tau} \sum_{i \in train} l_{\tau}(Y_i, f(X_i; \beta_{\tau})) + \lambda \mathcal{P}(\beta),$$

where $\mathcal{P}(\beta)$ to punish those samples against monotonicity.

Suppose we want to embed our model with some property (like translation, scale, etc. which are equation), we simply put them into loss function. However, we need to avoid such equation, for $\tau_1 = 0.1, \tau_2 = 0.5$, we need to make

$$f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_2}) < 0$$

difference as large as possible without violating the basic purpose to estimate the quantile. On the other hand, we could investigate positive part $X_+ = max(X, 0)$ as small as possible:

$$\begin{aligned}\mathcal{P}(\beta) = \lambda_1 \sum_{i \in train} \{[f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_2})]_+\}^2 + \lambda_2 \sum_{i \in train} \{[f(X_i; \beta_{\tau_2}) - f(X_i; \beta_{\tau_3})]_+\}^2 \\ + \lambda_3 \sum_{i \in train} \{[f(X_i; \beta_{\tau_1}) - f(X_i; \beta_{\tau_3})]_+\}^2,\end{aligned}$$

where $\lambda_{1,2,3}$ are positive number and $\lambda_1 + \lambda_2 + \lambda_3 = 1$. To make the loss small is to make positive part close to 0, which satisfies our requirement.

Code appendix

```
library('coda')
f <- function(x) ifelse(x >= 2, exp(-3*(x - 2)) + exp(-(x - 30)**2) + exp(-(x - 20)**2/0.01)
#Metropolis-Hasting Algorithm for random walk
rq <- function(x, sigma) rnorm(1, mean = x, sd=sigma)
mh <- function(x0, fun, rq, sg){
  n <- 1e6
  xall <- rep(NA,n+1)
  x <- x0
  xall[1] <- x


  for (t in seq(2,n+1)){
    x <- x
    y <- rq(x,sg)

    if (log(runif(1))<=log(fun(y))-log(fun(x))){
      x <- y
      }
    xall[t] <- x
    }
```

```
  return(xall)
}

set.seed(49)
num <- 3
x0 <- c(2, 20, 30)
sigma1 <- 1
sigma2 <- 5
sigma3 <- 8
chains1 <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma1)))
chains2 <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma2)))
chains3 <- lapply(1:num, function(i) mcmc(mh(x0[i], f, rq, sigma3)))
sigmas <- c(sigma1, sigma2, sigma3)
chain <- c(chains1, chains2, chains3)
for (i in 1:length(chain)){

  current_chain <- chain[[i]]

  ESS <- effectiveSize(current_chain)

  sigma_index <- floor((i - 1) / 3) + 1

  sigma_val <- sigmas[sigma_index]

  cat("Chain", i, ": Sigma is", sigma_val, ", ESS is:", ESS, "\n")
}
par(mfrow = c(1, 3))
traceplot(chains1, main = paste("Traceplot: sigma =", 1))
traceplot(chains2, main = paste("Traceplot: sigma =", 5))
traceplot(chains3, main = paste("Traceplot: sigma =", 8))
chains1 <- mcmc.list(chains1)
chains2 <- mcmc.list(chains2)
chains3 <- mcmc.list(chains3)

gelman.diag(chains1)
gelman.diag(chains2)
gelman.diag(chains3)
par(mfrow = c(1, 3))
gelman.plot(chains1, main = paste("Gelman plot: sigma =", 1), auto.layout =  FALSE)
gelman.plot(chains2, main = paste("Gelman plot: sigma =", 5), auto.layout =  FALSE)
gelman.plot(chains3, main = paste("Gelman plot: sigma =", 8), auto.layout =  FALSE)
library('truncdist')
```

```r
# Improved MH for support
set.seed(355)
MH <- function(f,init){
  n <- 1e6
  xall <- rep(NA,n)
  x <- init
  xall[1] <- x
  for (t in seq(2,n)){
    y <- rtrunc(n=1, 'norm', a=2, mean = x, sd = 2)
    if (runif(1)<=f(y)*pnorm(2, mean=x, sd = 2, lower.tail = FALSE)/(f(x)*pnorm(2, mean = y,
      x<-y
    }
    xall[t] <- x
    }
  return(xall)
}
par(mfrow = c(1, 2))
x1 = c(2, 20, 30)
chains4 <- lapply(1:3, function(i) mcmc(MH(f, x1[i])))
traceplot(chains4)
gelman.diag(chains4)
gelman.plot(chains4, auto.layout =  FALSE)
abline(h = 2, col = 'gray')
paralleltemp <- function(num.steps, Ts, sg, init, threshold) {
  M <- length(Ts)
  samples <- matrix(NA, nrow = num.steps, ncol=M)

  X <- rep(init, M)

  for (n in 1:num.steps) {
    if (runif(1) <= threshold){
      # we are going to do a x-step
      for (m in 1:M) {
        y <- X[m] + sg[m]*rnorm(1)
        alpha <- f(y)^(1/Ts[m]) / f(X[m])^(1/Ts[m])

        if (runif(1) <= alpha) {
          X[m] <- y
        }
      }
    }
       else{
```

```
        ### we are going for a swap

        m <- sample(1:(M-1), 1)

        num = (f(X[m])^(1/Ts[m+1])*f(X[m+1])^(1/Ts[m]))
        denom = (f(X[m])^(1/Ts[m])*f(X[m+1])^(1/Ts[m+1]))

        if (runif(1) <= num/denom) {
          X[c(m, m+1)] = X[c(m+1, m)]
        }
     }
        samples[n, ] = X
  }

  return(samples)
}
set.seed(80)
Ts <- c(1, 20, 100, 200, 500)
sigmas <- c(0.1, 0.5, 1, 2, 5)
nstep <- 100000
init <- c(2, 20, 30)
chains5 <- lapply(1:length(init), function(i) paralleltemp(nstep, Ts, sigmas, init[i], 0.5))
xnodes <- seq(2, 50, 0.1)
k <- integrate(f, lower = 2, upper = Inf)$value
#plot(chains5[1][,1], type='l')
par(mfrow = c(3, 2))
for (i in 1:length(chains5)) {
    chain_samples <- chains5[[i]][, 1]

    plot(chain_samples, type='l')

    hist(chain_samples,
         breaks = 50,
         probability = TRUE,
         main = paste("Chain", i, "Histogram (T1)"),
         xlab = "x",
         ylab = "Density",
         col = "lightblue",
         border = "white")

    lines(x = xnodes,
          y = k * f(xnodes),
```

```
          col = "red",
          lwd = 2)
}
par(mfrow = c(1, 2))
chain_T1_run1 <- mcmc(chains5[[1]][, 1])
chain_T1_run2 <- mcmc(chains5[[2]][, 1])
chain_T1_run3 <- mcmc(chains5[[3]][, 1])
T1_mcmc_list <- mcmc.list(chain_T1_run1, chain_T1_run2, chain_T1_run3)
traceplot(T1_mcmc_list)
gelman.plot(T1_mcmc_list, auto.layout = FALSE)
gelman.diag(T1_mcmc_list)
library("mcmcse")
xf <- function(x) x * f(x) / k
true <- integrate(xf, lower = 2, upper = Inf)
ci <- function (data, alpha = 0.95){
  ci <- c()
  mean <- mean(data)
  sd <- sd(data)
  q <- qnorm((1 - alpha)/2)

  ci <- c(mean + q * sd, mean - q * sd)
  ci
}
ci_plot <- function(data, title){
  N <- length(data)
  sd.est <- sd(data)
  sd.est.CV <- sd.est
  running.mean <- cumsum(data)/seq(1, N)
  delta.ci <- 1.96*sd.est/sqrt(seq(1, N))
  #Plot the running mean and confidence interval
  plot(seq(1, N), running.mean, type='l', ylim = c(2,30), main = title)
  lines(seq(1, N), running.mean + delta.ci , col='blue')
  lines(seq(1, N), running.mean - delta.ci , col='yellow')
  abline(h = true$value, col = 'red')
}
par(mfrow = c(1, 3))
set.seed(42)
#From 1(a)
chain1_d <- mh(15, f, rq, 1)
#From 1(b)
chain2_d <- MH(f, 15)
#From 1(c)
```

```r
chain3_d <- paralleltemp(1e5, Ts, sigmas, 2, 0.5)[,1]

cat("The estimator by chain 1 is:", mean(chain1_d), "and ci is:", "[", mean(chain1_d)-1.96*sd
cat("The estimator by chain 2 is:", mean(chain2_d), "and ci is:", "[",mean(chain2_d)-1.96*sd
cat("The estimator by chain 3 is:", mean(chain3_d), "and ci is:", "[",mean(chain3_d)-1.96*sd


ci_plot(chain1_d, "RW, sigma=1")
ci_plot(chain2_d, "Truncated proposal")
ci_plot(chain3_d, "Parallel tempering")


library('MASS')
set.seed(46)
data <- data.frame(y = Boston$medv,
                   x1 = Boston$lstat,
                   x2 = Boston$rm)

n <- length(data$y)

tr_index <- sample(1:n, floor(n * 0.7))


data_tr <- data[tr_index, ]

data_test <- data[-tr_index,]

loss <- function(tau, y, q){
  ifelse(y > q, tau * (y - q), (tau - 1) * (y - q))
}

empirical_loss <- function(beta, tau, x1, x2, y){
  beta1 <- beta[1]
  beta2 <- beta[2]
  beta3 <- beta[3]


  q <- beta1 + beta2 * x1 + beta3 * x2

  return(mean(loss(tau, y, q)))

  }
```

```r
init_value <- c(0, 0, 0)
tau <- c(0.1, 0.5, 0.9)
beta_es <- matrix(data = 0, nrow = 3, ncol = 3)

for (i in 1 :length(tau)){
  opt_result <- optim(par = init_value, fn = empirical_loss, tau = tau[i], x1 = data_tr$x1,
  x2 = data_tr$x2, y = data_tr$y, method="Nelder-Mead")
  beta_es[i, ] <- opt_result$par
  cat("When tau is", tau[i], ",the empirical loss is", opt_result$value, "and the estimator i

}
matrix_test <- matrix(data = 1, nrow = length(data_test$y), 3)
matrix_test[,2] <- data_test$x1
matrix_test[,3] <- data_test$x2

matrix_y <- matrix(data = 1, nrow = length(data_test$y), ncol = length(tau))
for (i in 1:3){
  matrix_y[,i] <- data_test$y
}
fx <- matrix_test %*% t(beta_es)

for (i in 1:length(tau)){
  los <- empirical_loss(beta_es[i,], tau[i], data_test$x1, data_test$x2, data_test$y)
  cat("When tau is", tau[i], ",the empirical loss is:", los, "\n")
}
compare <- matrix_y <= fx
count_y_smaller_fx <- colSums(compare)/length(data_test$y)

count_y_smaller_fx

empirical_risk_b <- function(beta, tau, x1, x2, y){
  beta1 <- beta[1]
  beta2 <- beta[2]
  beta3 <- beta[3]
  beta4 <- beta[4]
  beta5 <- beta[5]
  beta6 <- beta[6]


  q <- beta1 + beta2 * x1 + beta3 * x2 + beta4 * x1 ** 2 + beta5 * x2 ** 2 + beta6 * x1 * x2

  return(mean(loss(tau, y, q)))
```

```
   }
init_value_b <- c(0, 0, 0, 0, 0, 0)
tau <- c(0.1, 0.5, 0.9)
beta_es_b <- matrix(data = 0, nrow = 3, ncol = 6)
#beta_es_b

for (i in 1 :length(tau)){
  opt_result_b <- optim(par = init_value_b, fn = empirical_risk_b, tau = tau[i], x1 = data_tr
  x2 = data_tr$x2, y = data_tr$y, method="Nelder-Mead")
  beta_es_b[i , ] <- opt_result_b$par
  cat("When tau is", tau[i], ",the empirical loss is", opt_result_b$value, "and the estimator
  opt_result_b$convergence
}

matrix_test_b <- matrix(data = 1, nrow = length(data_test$y), ncol = 6)
matrix_test_b[,2] <- data_test$x1
matrix_test_b[,3] <- data_test$x2
matrix_test_b[,4] <- matrix_test[,2] **2
matrix_test_b[,5] <- matrix_test[,3] **2
matrix_test_b[,6] <- matrix_test[,2] * matrix_test[,3]

fx_b <- matrix_test_b %*% t(beta_es_b)

for (i in 1:length(tau)){
  los_b <- empirical_loss(beta_es_b[i,], tau[i], data_test$x1, data_test$x2, data_test$y)
  cat("When tau is", tau[i], ",the empirical loss is:", los_b, "\n")
}

compare_b <- matrix_y <= fx_b
count_y_smaller_fx_b <- colSums(compare_b)/length(data_test$y)
count_y_smaller_fx_b


set.seed(46)
lambda_list <- seq(1e-4, 1e-1, 0.001)
k_fold <- 5
folds <- sample(rep(1:k_fold, length.out=length(data_tr$y)))
empirical_risk_c <- function(par, lambda, tau, x1, x2, y){
  beta1 <- par[1]
  beta2 <- par[2]
  beta3 <- par[3]
  beta4 <- par[4]
```

```
  beta5 <- par[5]
  beta6 <- par[6]


  q <- beta1 + beta2 * x1 + beta3 * x2 + beta4 * x1 ** 2 + beta5 * x2 ** 2 + beta6 * x1 * x2

  return(mean(loss(tau, y, q)) + lambda * sum(par**2))
}
best_lambda <- function(tau){
  init_value_c <- rep(0, 6)
  cv_mse <- numeric(length(lambda_list))
  convergence <- c()

  for (j in 1: length(lambda_list)){
      errs <-numeric(k_fold)

      for (i in 1:k_fold){
        tr <- folds != i
        ts <- folds == i

        tr_da <- data.frame(y = data_tr$y[tr], x1 = data_tr$x1[tr], x2 = data_tr$x2[tr])
        ts_da <- data.frame(y = data_tr$y[ts], x1 = data_tr$x1[ts], x2 = data_tr$x2[ts])

        opt_result_c <- optim(par = init_value_c, fn = empirical_risk_c, lambda = lambda_list

        par <- opt_result_c$par

        err <- empirical_risk_c(par, lambda = 0, tau, ts_da$x1, ts_da$x2, ts_da$y)

        errs[i] <- err
        #cat(par, "\n")

        #cat("Convergence:", opt_result_c$convergence, "\n")
        convergence <- c(convergence, opt_result_c$convergence)
      }
      ms <- mean(errs)
      cv_mse[j] <- ms

  }
  cat("About", 100 - 100 * mean(convergence), " percent of experiments converge in the traini
  return(cv_mse)
}
```

```r
lambda_test <- numeric(length(tau))
for (i in 1:length(tau)){
  min_index <- which.min(best_lambda(tau[i]))
  min_lambda <- lambda_list[min_index]
  lambda_test[i] <- min_lambda
  cat("When tau is", tau[i], ",","the best lambda is", min_lambda, "\n")
}
beta_es_c <- matrix(0, nrow =6, ncol = 3)
init_value_c <- rep(1, 6)
for (j in 1: length(lambda_test)){

      opt_result_c2 <- optim(par = init_value_c, fn = empirical_risk_c, lambda = lambda_test

      beta_es_c[,j] <- opt_result_c2$par
      cat("The empirical loss is", opt_result_c2$value, "the coefficients are", opt_result_c2

}
for (i in 1:length(tau)){
  los_b <- empirical_risk_b(beta_es_c[,i], tau[i], data_test$x1, data_test$x2, data_test$y)
  cat("When tau is", tau[i], ",the empirical loss is:", los_b, "\n")
}

fx_c <- matrix_test_b %*% beta_es_c

compare_c <- matrix_y <= fx_c
count_y_smaller_fx_c <- colSums(compare_c)/length(data_test$y)
count_y_smaller_fx_c
```